Multi View Optical Only Drone Tracking

Joaquin Manuel Philco Montoya

November 12, 2025

Abstract

Design and propose a pipeline to detect and track the 3D position (x, y, z with respect to the camera) of a single flying drone using images from a ground visual sensor system. The drone may change altitude, distance, and orientation, and may move across the sky with varying backgrounds and lighting.

1 Problem Breakdown

When dealing with a tracking problem of this kind in which depth perception is needed, it would be best to split the problem into the following smaller sub problems.

1. **Calibration of Cameras** - This will allow us to recover information about our setup like intrinsics matrices for each cameras or other parameters like transformations from one camera frame to another if a multi-view architecture is used.
2. **Feature Detection** - For finding the location of the drone (x, y) within the image itself in the pixel coordinates.
3. **Depth Perception** - To find the location of Drone z in meters with respect to the world's origin.
4. **Pose Estimation to simplify computation on next steps** - Once we have an initial location both in pixel coordinates and depth. We can utilize optical flow to predict the location in which the drone would be in the vicinity of pixels. This would allow for reduced computation for feature detection algorithms.

Now that we have the sub-set of problems we can find solutions for each of these. Moreover, we can optimize each of the solutions depending on the setup and challenge that we have. Optimization will be a game of tug-of-war between the goals, setup and constraints of the problem.

2 Problem Constraints

- **Optical-only**: no IMU, GPS, or odometry. Stationary rig; the drone moves within the FOV.
- **Calibration available**: Each camera's intrinsic matrix and lens distortion are known (from checkerboard). For stereo, the extrinsics (relative R, t) are known or can be retrieved.
- **Synchronization**: Camera hardware's may br different from each other. This means that there is no guarantee of synchronized frames. We will (i) prefer hardware trigger/PTP when possible; otherwise (ii) time-align pairs by nearest timestamp.
- **Output frame**: all 3D positions (x, y, z) are reported in the *left camera frame* unless stated otherwise or camera 0.
- **Environmental Context**: Background is sky/urban skyline with varying illumination.
- **Single target**: exactly one drone in view.

3 System Setup

Since we are dealing with a tracking problem. It would be best to use a multi-view camera setup. Moreover, we will evaluate a basic configuration of 2 cameras and we will increase the number of them depending on performance gains.

3.1 Two Cameras

3.1.1 Base Case (Parallel Setup)

- 2 parallel cameras separated by a baseline of 1 meter.
- Each cameras will be on a stand of about 1 meter as well.
- Each camera intrinsics and extrinsics matrices will be known.
- Ideally, the cameras will be synchronized.
- We maintain a standard coordinate frame based on the left camera.
- Stationary camera setup, the drone is a moving object within the field of view.
- Prefer global shutter. If rolling shutter is used, constrain exposure to limit motion distortion. Exposure/white balance are locked across cameras.

A setup like this one will simplify the viewing geometry of the problem. Since this one is known, a lot of assumptions and simplification can be done for the tracking pipeline

3.1.2 Known Orientation (Calibrated, Non-parallel)

- Cameras are not parallel, but relative pose (R, t) is known from stereo calibration.
- Either (a) *rectify* using (R, t) and proceed as in the base case, or we can (b) triangulate directly with $P_L = K_L[I|0]$, $P_R = K_R[R|t]$.

This is a more realistic setup in which the camera positions are not parallel. Although knowing the extrinsics of this one allows us to find a fundamental transformation from one pixel coordinate frame to another.

3.1.3 Unknown Orientation (Uncalibrated at Deployment)

- Intrinsics are known; the inter-camera pose (R, t) is unknown initially.
- Option A: Run a quick on-site stereo calibration to recover (R, t), then use the known orientation path mentioned before.
- Option B (if calibration board unavailable): self-calibrate from background features via Essential matrix plus cheirality to get R and direction of t. Scale is unknown until set by a measured baseline B or a known-size/ground-plane constraint; after setting scale, rectify/triangulate as usual.
- Requires sufficient static background parallax; pure rotation or low texture is degenerate.

3.2 Multi-View 2+ Cameras

Having a multi-view setup in which there is more than 2 cameras allows us to:

- Reduce problems with occlusion
- Reduce dependency on one camera if this one fails. If the systems looses a cameras, having more than 2 would allow for the system to remain.

4 Camera Calibration

If we are in the field and we are allowed to perform calibration of the cameras before the drone starts moving, then we need to calibrate our cameras to retrive the following important charactertistics:

1. Camera Intrinsics
2. Camera Extrensics
3. Fundamental Matrix
4. Essential Matrices Between Cameras
5. Distortion coefficients for image aberrations
6. Baselines and/or translations between cameras
7. Stereo Rectification Matrices

4.1 Camera Intrinsics

Definition: Internal camera parameters such as focal length per pixel unit and principal point that allows for projecting points in image coordinates to pixel coordinates (space in which we are going to be plotting the tracking of the drone).

Retrieval: Checkerboard calibration can be performed by taking multiple pictures of this one at different orientations to find these calibration targets. OpenCV's `calibrateCamera()` can be used in to this set of pictures for estimating the intrinsic matrix K.

4.2 Camera Extrinsics

Definition: Rotation and translation of each camera relative to a reference coordinate system which in this case is the first camera frame (world coordinate).

Retrieval: For retrieving extrinsics, it depends on whether the system would be a stereo based one and on whether the cameras share some of their field views. If the setup is a stereo one, the extrinsics can be retrieved during stereo calibration and using opencv functions like `stereoRectify()`. Nonetheless, if the setup presents a more than 2 cameras, well, i know there are some methods to do so if the cameras share the same field of view. Neverthelss, I am not truly that familiar with them and it gets worse if they do not share FoV.

4.3 Fundamental Matrices

The Epipolar constriant:

$$\mathbf{x'}^T F \mathbf{x} = 0$$

where $\mathbf{x}$ and $\mathbf{x'}$ are in homogeneous pixel coordinates. This matrix captures the mapping of points to their corresponding epipolar lines without requiring knowledge of the cameras' internal parameters.
Retrieval: The fundamental matrix can be estimated directly from a set of point correspondences between two images using methods such as the eight-point algorithm or RANSAC-based robust estimation. Nevertheless, OpenCV's `findFundamentalMat()` can be used.

4.4 Essential Matrices

Definition: Encodes the relative rotation and translation between two calibrated cameras:

$$E = [t]_\times R$$

where $[t]_\times$ is the skew-symmetric matrix of the translation vector.

Retrieval: Once intrinsics (K) are known and the fundamental matrix F is computed from point correspondences, the essential matrix is derived as:

$$E = K^T F K$$

4.5 Distortion Coefficients

Definition: Corrects lens distortions such as radial-barrel distortions (k_1, k_2, k_3) and tangential distortion (p_1, p_2).

Retrieval: These parameters can be retrieved in the same operation as the one that retrieved the intrinsic matrix for each lens, since the parameters are different for each lens.

4.6 Baseline and/or Translation

Definition: Is the relative translation vector between two cameras.

Retrieval: The translation vector t is directly estimated during stereo calibration.

4.7 Stereo Rectification Matrices

Definition: Rotation and projection matrices that warp stereo image pairs so that epipolar lines are horizontal.

Retrieval: Once R and t are known from stereo calibration, OpenCV's `stereoRectify()` computes R_1, R_2, P_1, P_2 and Q. These are then used with `initUndistortRectifyMap()` and `remap()` to rectify live stereo streams.

5 Feature Detection (Drone Detection)

5.1 Overview

Given an image stream $\{I_t\}$ from each camera, we first compute the optical flow to obtain an approximated motion field (since Lucas-Kanade uses the brightness consistency assumption). We then place fixed-size candidate windows centered at local maxima of motion magnitude and run a lightweight detector (CNN/ViT) only within those windows. Finally, The detector outputs an in-window drone location that can be mapped back to full-frame pixel coordinates.

5.2 Step 1: Computing the optical flow as a prior

Compute an approximated optical flow between consecutive frames for example using Lucas-Kanade or RAFT. Let $\mathbf{v}_t(u, v)$ be the flow at pixel (u, v), and $m_t(u, v) = \|\mathbf{v}_t(u, v)\|$ its magnitude. We can find local maxima subject to:

- naively magnitude threshold $m_t(u, v) > \tau_m$,
- non-maximum suppression radius r_{nms} to avoid clustered peaks,

5.3 Step 2: Compute the local maximum optical flow windows

For each selected peak (u_k, v_k), crop a square window W_k of side s (default $s = 200$ px for example but this is adaptive with scale or image resolution). Use the original-resolution frame for the crop, even if flow was computed at a downsampled resolution. We can maintain a maximum number of windows per frame $K_{\max}$ to bound latency, this can depend on compute power available and if its needed to be live or not.

5.4 Step 3: In-Window Classification/Localization.

Run a small-object-tuned detector on each W_k:

- *CNN:* YOLOv8-nano / YOLOX-tiny fine-tuned on drone datasets (single-class "drone"), with anchor priors adapted to the drone size distribution.
- *ViT:* tiny DETR/YOLOS/DeiT-S for robust context modeling if GPU budget allows.

The detector outputs a bounding box $\mathcal{B}_k$ (and center $\hat{c}_k$) in window coordinates. Map back to image coordinates by adding the window offset and scaling if the window was resized for inference.

These steps will allow us to find the approximated pixel location within each frame of all the cameras. Now, this is extremely dependent on brightness changes and occlusion of the drone. For this, we can try to fix this with weighting each camera's detected output based on a confidence matrix and choosing the best output with an algorithm

6 Depth Perception

6.1 Stereo Setup

Depth perception can be achieved by triangulating corresponding pixel locations across two images of the same scene. Moreover, this is dependent on the setup that we have. In a common stereo setup, the parallax would be our most important variable that will determine how far away from the camera can we safely detect depth. If the parallax allows us, in a stereo setup, the depth can be Depth perception of the drone can easily be detected through any kind of triangulation of the pixel location where the drone is in each picture.estimated through a disparity to depth transformation of the rectified images from the stream. Moreover, since we know the essential matrices, the rectifying matrices, and disparity to depth matrix, we can create a transformation that allows us to take the live input from the cameras and determine a depth.

In practice, once the cameras are calibrated, the following steps allow for metric depth estimation:

1. **Rectification:** Using the essential matrix, extrinsics, and rectification matrices, the stereo pair is warped so that epipolar lines are aligned horizontally.

2. **Disparity Estimation:** For each pixel in the left image, the corresponding pixel is searched along the same row in the right image. The displacement between the two is the *disparity*.

3. **Depth Estimation:** The disparity is converted into depth using the disparity-to-depth matrix Q obtained from `stereoRectify()`, or equivalently by since the rectifying allows for the case of a parallel setup:
$$Z = \frac{f \cdot B}{d}$$
where Z is depth, f is the focal length, B is the baseline (distance between cameras), and d is the disparity.

6.2 Multiview Setup

If we have the orientation of each camera with respect to a frame of reference we can:

1. Find the drone's pixel location.

2. Using the intrinsic matrix K_i and extrinsic parameters (R_i, t_i), back-project the pixel into a normalized direction vector in the world frame:
$$\mathbf{d}_i = R_i^T K_i^{-1} \mathbf{x}_i$$

3. Normalize this new direction vector.

Doing this for each camera will allows us to find a normalized ray pointing, in world coordinate system, towards the drone's location. Utilizing this we can try to find the intersection or closest point between the projected normals. The Z component of this point would be the depth of the drone within the world coordinate system.

7 Algorithm

The steps mentioned beforehand can be utilize with a minimum of 2 cameras. The main advantage of using multiple viewpoints is that the algorithm can evaluate the quality and reliability of each detection by considering geometric and perceptual factors such as occlusion, parallax, and field-of-view overlap. We can achieve this by givin a confidence coefficient to each produced output based on any 2 cameras. This, tied up to a feedback loop can allows us to refine for the most useful points.

Therefore, Eeach pair of cameras (C_i, C_j) produces an independent estimate of the detected object's 3D position through triangulation or disparity-based reconstruction. Given N cameras, the total number of possible stereo pairs is

$$\frac{N(N-1)}{2},$$

each yielding an estimate $\mathbf{P}_{ij}$ and an associated confidence coefficient γ_{ij}.

The confidence coefficient can be modeled as a function of several quality indicators:

$$\gamma_{ij} = f(\text{baseline length}, \text{viewing angle}, \text{occlusion score}, \text{optical flow magnitude}, \text{image sharpness})$$

7.1 Pairwise Confidence Function

For the sake of simplicity since this can be fleshed out more, we are only going to focus on three common factors:

1. Baseline (b_{ij}). A longer baseline generally increases triangulation accuracy, as the disparity between image projections becomes more pronounced for a given scene depth. However, if the baseline is excessively large relative to the target's distance, the overlap of the fields of view may decrease, leading to fewer common keypoints and higher matching errors. Typical effective baselines for mid-range stereo setups (1–10 m target distance) range from 0.05 m to 0.25 m. In this system, higher baseline values are rewarded with higher confidence up to a soft saturation point b_0, beyond which improvements diminish.

2. Overlapping Field of View (ϕ_{ij}). A large overlap ensures that more common features are available for correspondence matching and triangulation, whereas small overlaps increase the likelihood of partial visibility and depth ambiguity. We define ϕ_{ij} to be a value in between 0 and 1, where 1 corresponds to full overlap and 0 indicates no shared visibility region. Empirically, overlaps above 60% provide stable reconstructions, while below 30% confidence should be penalized sharply since disparity estimates become unreliable.

3. Occlusion Score (o_{ij}). Occlusion can happen due to many things like objects in a scene or self-occlusion of the object itself. We define the occlusion score to be a float number from 0 to 1, where 0 indicates full visibility and 1 indicates complete occlusion. Even partial occlusions reduce the number of valid correspondence points which can be detrimental for the accuracy of triangulation. For this reason, we can make the confidence function penalizes higher occlusion scores, typically raising $(1 - o_{ij})$ to a power $\alpha \in [1, 3]$ to emphasize its effect.

For a camera pair (C_i, C_j), let b_{ij} be the baseline length (m), $\phi_{ij} \in [0,1]$ the overlapping field of view ratio (0 = no overlap, 1 = full overlap), and $o_{ij} \in [0,1]$ an occlusion score (0 = clear, 1 = fully occluded).
We map each raw factor to a bounded quality score from 0 to 1:

$$g_{\text{base}}(b_{ij}) \;=\; \frac{b_{ij}}{b_{ij} + b_0}, \qquad g_{\text{overlap}}(\phi_{ij}) \;=\; \phi_{ij}^{\beta}, \qquad g_{\text{occ}}(o_{ij}) \;=\; (1 - o_{ij})^{\alpha},$$

where $b_0 > 0$ sets a soft saturation for the baseline effect, $\beta \geq 1$ adjusts the sensitivity to limited overlap, and $\alpha \geq 1$ controls how sharply occlusion is penalized.

The overall pairwise confidence is obtained as the product of the three components:

$$\gamma_{ij} \;=\; g_{\text{base}}(b_{ij}) \cdot g_{\text{overlap}}(\phi_{ij}) \cdot g_{\text{occ}}(o_{ij}),$$

Parameter guidance. Set b_0 near the average baseline of the camera array so that shorter baselines are mildly penalized; choose $\beta \in [1,3]$ to reward high FoV overlap and suppress marginal ones; and $\alpha \in [1,3]$ to make the confidence drop more aggressively under occlusion.

7.2 Weighted Fusion of Multi-Camera Estimates

To obtain a unified and consistent position estimate, all pairwise detections are fused through a weighted averaging process:

$$\mathbf{P}^* = \frac{\sum_{i<j} \gamma_{ij} \, \mathbf{P}_{ij}}{\sum_{i<j} \gamma_{ij}}$$

where $\mathbf{P}^*$ represents the refined 3D location that maximizes overall confidence across all views.

7.3 Feedback Loop for Adaptive Refinement

A feedback mechanism can be introduced to iteratively refine the confidence model and detection accuracy. The loop operates as follows:

1. **Initial Detection:** Each camera pair generates a preliminary detection and confidence value.
2. **Fusion:** Weighted fusion combines all pairwise results into a global estimate.
3. **Error Analysis:** The reprojection error across all cameras is measured by projecting $\mathbf{P}^*$ back into each image plane.
4. **Confidence Update:** The confidence coefficients γ_{ij} are updated based on the reprojection residuals and temporal stability over successive frames.
5. **Iteration:** The updated confidence values are used in the next iteration to prioritize more reliable viewpoints.

So far with this, theoretically, we have created a feedback loop that allows for the system to update its coefficients of confidence. This will allow for the system to favor camera pairs that consistently provide more accurate outputs that minimize problems with occlusions. Moreover, since the confidence coefficients are updated, this would make the system stable to lighting conditions and motion blur.

7.4 Possible Upgrades

Well, this is a feedback loop that utilizes most of spatial, geometric and image quality markers to dictate the confidence of the detections. Something very important when dealing with moving objects is how we can adjust for the temporal domain. For this, algorithms like the extended Kalman filter could benefit this.

8 Afterword

This afterword exists just to explain a couple of the things and decision i took over here. To be honest, this is the first time I have dealt with multiview architectures. I am more confident in using perspective geometry in a stereo setup rather than with multiple cameras. Some models are too computational hungry like ViTs which are just not scalable in an open real world scenario. If we look at the dataset example you have provided me with. I noticed some embedded devices like mobile phones within it. Although current phones nowadays have NPUs for specific ai tasks, these still are limited and not available widespread. I feel like using a combination of both perspective geometry and neural network models would be the best bet to reduce computational consumption without sacrificing much of a latency. Moreover, some current smartphones also produce their own disparity maps by splitting their pixel sensor at the camera level itself. This creates a small parallax that helps with creating a small approximation of depth. This is normally how portrait mode works in a phone. Nevertheless, not all devices have it. I mention this because system would be extremely dependent on the available hardware. The problem provided with a very broad overview of hardware "optical sensors" and that is it. This lead me to just make a stereo setup as mentioned in the section 3 of this paper. Nevertheless, the dataset has multiple view camera set and a well define hardware setup. I did not know whether to follow the constraints of the problem or the dataset so for that i answer both problems. For the problem stated, a simple binocular system would suffice to detect and find the drone's 3D position. Although this one would be limited by its field of view and sensible to occlusion.
Regardless, this was fun.